

Assignment 10

Problem 0a. Alphabet Dictionary

Given the following string, write a program that counts **how many times each letter or number** has been used in the string. Use a **dictionary** to store your results.

Enter a phrase: "The lazy dog jumped over the curious cat"

Here's a sample running of your program:

Report in ascending order by ASCII value:

Enter a phrase: "The lazy dog jumped over the curious cat"

```
7
T 1
a 2
c 2
d 2
e 4
g 1
h 2
i 1
j 1
l 1
m 1
o 3
p 1
r 2
s 1
t 2
u 3
v 1
y 1
z 1
```

Note this example only includes letters. Numbers would be placed, from 0-9, before the letters and after the spaces.

Problem 0b

In the example above, the t's are counted as separate letters. We want to count lowercase and uppercase letters as the same and remove all characters that are not alphanumeric.

Begin by writing this convenience function - this function will be useful to make sure that all of the string data you will be working with is formatted correctly.

```
# function:   cleanup_string
# input:      a string to clean up
# processing: (1) makes the entire string lowercase.
#             (2) retains only alphabetic and numeric characters
#             all punctuation, spaces, and special characters removed]
# output:     returns the cleaned up string
def cleanup_string(data):

# TEST CODE
test1 = cleanup_string("Hello World! This is a simple test of this function!"
)
print (test1)
# helloworldthisisasimpletestofthisfunction

test2 = cleanup_string("ABC123abc this is Another TEST!!!#@")
print (test2)
# abc123abcthisisanothertest
```

In part a, the string would be converted to

```
thelazydogjumpedoverthecuriouscat
```

and the output would be

Report in ascending order by ASCII value:

```
a 2
```

```
c 2
d 2
e 4
g 1
h 2
i 1
j 1
l 1
m 1
o 3
p 1
r 2
s 1
t 3
u 3
v 1
y 1
z 1
```

Problem 0 Points (15):

- Dictionary is used to track how many of each letter; letters are keys, counts are values (3 points)
- Output of number and letter counts is sorted in ascending order (2 points)
- Two test cases that ensure that any non-alphanumeric characters are removed and letters and numbers are correctly counted (5 points each; 2 points each for the cleaned string, 3 points each for the sorted counts)

Problem 1

Note that the programs you will be writing build on one another. You will want to attempt these parts in order. You can also feel free to save all of your work in one big file (call it "LastNameFirstName_assign10_part1.py")

For this project you will be re-writing the "product database" program from Assignment 08 using dictionaries. This will help you to compare and contrast how dictionaries and lists operate in similar yet different ways. You'll also notice that some tasks tend to be much easier (and more efficient!) to write with dictionaries, while others may be more challenging.

To start copy the following dictionary into your program:

```
inventory = {  
    'soft drink': [0.99, 10],  
    'onion rings': [1.29, 5],  
    'small fries': [1.49, 20]  
}
```

Each 'key' in the dictionary is a product (string). Each 'key' has a value (list) which contains the price of that product in position 0 and the amount of that product in position 1.

You can access items in the dictionary using two sets of square brackets. For example:

```
print ("soft drinks cost:", inventory["soft drink"][0])  
print ("we have:", inventory["soft drink"][1], "soft drinks at our store")
```

Your task is to add the following functionality to this program:

- Overarching menu system
- Search for a product
- List all products
- Add a product
- Remove a product
- Update a product
- Report on all products

Here's a sample running of this program (user input is underlined):

```
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: foobar  
Unknown command, try again  
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: l  
Product           Price      Quantity  
onion rings        1.29       5  
small fries        1.49      20  
soft drink         0.99      10  
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: s  
Enter a product name: soft drink  
We sell 'soft drink' at 0.99 per unit  
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: s  
Enter a product name: apples  
Sorry, we don't sell 'apples'  
(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: a
```

Enter a product name: soft drink

Duplicate product name

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: a

Enter a product name: apples

Enter a price: -10

Invalid range, try again

Enter a price: 3.50

Enter an inventory amount: -10

Invalid range, try again

Enter an inventory amount: 150

Invalid range, try again

Enter an inventory amount: 75

Product added

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: l

Product	Price	Quantity
apples	3.50	75
onion rings	1.29	5
small fries	1.49	20
soft drink	0.99	10

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: r

Enter a product name: foobar

Unknown product name

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: r

Enter a product name: small fries

Product removed

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: l

Product	Price	Quantity
apples	3.50	75
onion rings	1.29	5
soft drink	0.99	10

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: u

Enter a product name: apples

(n)ame, (p)rice or (a)mount? p

Enter a new price: 2.50

Price updated

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: u

Enter a product name: apples

(n)ame, (p)rice or (a)mount? n

Enter a new name: soft drink

Product name already exists

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: u

Enter a product name: apples

(n)ame, (p)rice or (a)mount? n

Enter a new name: pears

Product renamed

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: l

Product	Price	Quantity
onion rings	1.29	5
pears	2.50	75
soft drink	0.99	10

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: e

Total cost of all items in inventory: 203.85

Highest priced item: 2.5 is pears

Lowest priced item: 0.99 is soft drink

(s)earch, (l)ist, (a)dd, (r)emove, (u)pdate, r(e)port or (q)uit: q

See you soon!

Here are some hints on each part:

- You may use any general code you that you wrote for assignment #8 (menu system, etc.)
- The 'SEARCH' feature should be fairly straightforward - the KEY in the dictionary is the name of the product. How can you test to see if a KEY is in a dictionary?
- The 'LIST' feature should also be fairly easy. How do you iterate over all KEYS in a dictionary?
- The 'ADD' feature should be easy as well. How do you add a new KEY / VALUE pair to a dictionary? What are you storing as a value to match the sample data given in the dictionary above? You need to make sure that the item is not in the dictionary already as a KEY before you add it. Note that prices must never fall below \$0.00. Inventory must never fall below 0 or go above 100.
- The 'REMOVE' feature should be straightforward. How do you delete a KEY / VALUE pair from a dictionary?
- 'UPDATE' should be fairly easy as well. How do you change a VALUE in a dictionary if you know the KEY associated with it?

- 'REPORT' will be the trickiest function to write. For this one you will need to find three things:
 - The **total cost of all items**: This will require that **you visit all items in** the dictionary and maintain some kind of **accumulator**
 - The highest priced item and the lowest priced item may be tricky since the 'max' function will not work on a dictionary. **You may not use any technique not taught in class to do this - i.e. writing a 'lambda' function.** Hint - you will need to implement the **'maximum' / 'minimum' algorithm yourself** which involves the following:
 - Assume the first item in the list is the biggest / smallest
 - Visit all other items in the list and determine if they are bigger / smaller than the maximum / minimum you already know about
 - At the end of your investigation you **will have found the biggest / smallest**

Problem 1 Points (40)

Note: a **dictionary structure** must be used in order to receive credit for this problem.

2.5 points each (in some of these, multiple cases will be tested, adding up to 16 tests/40 points):

- Invalid options not allowed
- Searching for a product that **does not exist results** in "Sorry, we don't sell [product name]"
- Searching for a product that **does exist results in** printing the product name, price, and quantity
- Products can be added with **proper cost and quantity (i.e. positive values)**
- Products that are already in the inventory cannot be added
- Invalid costs are not accepted when adding products
- Invalid quantities are not accepted when adding products
- Products that aren't in inventory can't be removed
- Products that are in inventory can be removed
- Products that aren't in inventory can't be updated
- Products that are in inventory can have cost updated
- Products that are in inventory can have quantity updated
- When products are reported, the most and least expensive items/costs are printed, and the sum of all products is printed
- 'q' input causes the program to end

Problem 2. Battleship

Note that the programs you will be writing build on one another. You will want to attempt these parts in order. You can also feel free to save all of your work in one big file (call it "LastNameFirstName_assign10_part2.py")

In this program you will be using nested lists to create a game of Battleship between a user and the computer.

Problem 2a

First, you will be creating two gameboards. They will be 4x5, and one will be for the user and one will be for the computer. To represent that spaces have not yet been chosen, they can be filled in with '.'s. Each game board should look like this when printed (the ---- are not part of the board but should be printed after every row).

```
.....  
-----  
.....  
-----  
.....  
-----  
.....  
-----
```

Problem 2b

You will then use a series of input functions to prompt the user to place 5 ships. In your user board, mark these locations with an 'S'. First ask the user for a row number, then a column number. Validate that they are entering a row between 0 and 3 and a column between 0 and 4. Validate that they have not already placed a ship in that location. Let the user know when they place a ship. When they have placed all 5 ships, print out the board.

Example:

```
Please enter a row: 0  
Please enter a column: 1  
Ship 1 placed.  
Please enter a row: 4  
Invalid, try again!  
Please enter a row: 2  
Please enter a column: -1  
Invalid, try again!  
Please enter a column: 2  
Ship 2 placed.  
Please enter a row: 0  
Please enter a column: 1  
Ship already exists in that location.
```



```

Please enter a row: 0
Please enter a column: 3
Ship 3 placed.
Please enter a row: 3
Please enter a column: 1
Ship 4 placed.
Please enter a row: 1
Please enter a column: 4
Ship 4 placed.
Here is your board:
.S.S.
-----
....S
-----
..S..
-----
.S...
-----

```

Problem 2c

Add 5 ships to the computer's board. Do this using the `random.randint` function. For each of the 5 ships, randomly select the row (0-3) and column (0-4) numbers, validate that there is not already a ship in that location, and update the board appropriately.

Problem 2d

Now you have everything you need to play the game. After both boards are set up, the player and computer will take turns trying to "hit" each other's ships. After the user or computer takes their shot, report if they hit or missed the ships. At the beginning and end of each turn, the current board will be printed. 'X' represents a hit, 'O' represents a miss, and '.' are used to mark spaces that have not yet been visited. For the computer's board, you will need to "hide" their ships so the user cannot see them. Therefore, when you print the computer's board, if you encounter an 'S', print a '.' instead.

When all 5 ships are hit by either the user or the computer, end the game (i.e. using a while loop). When the game ends, report the winner. Also report the number of turns – you can count the number of times the user has gone (since if the user wins, the computer will have had one fewer turn).

Remember, you will need to **validate the input in the** same way as above (make sure they are entering a valid row/column number and are not going to a previously visited location – marked by 'X' or 'O').

Here is an example of how a few turns would go, using the example gameboard shown above:

Let's play!

User's turn...

Please enter a row: 0

Please enter a column: 1

Miss!

Computer's board:

.0...

.....

.....

.....

Computer's turn...

Miss!

User's board:

.S.S.

.0..S

..S..

.S...

User's turn...

Please enter a row: 0

Please enter a column: 1

You've already gone here, please try again.

```
User's turn...
Please enter a row: 1
Please enter a column: 1
Hit!
Computer's board:
.0...
-----
.X...
-----
.....
-----
.....
-----
```

```
Computer's turn...
Hit!
User's board:
.S.X.
-----
.0..S
-----
..S..
-----
.S...
-----
```

And an example of how the game would end:

```
User's turn...
Please enter a row: 3
Please enter a column: 4
Hit!
Computer's board:
.0..X
-----
```

.X..X

O.X..

.OO.X

User won in 9 turns!

Problem 2 Points (45):

- Nested 2D list is used to hold game boards – one for user and one for computer; list has 4 rows and 5 columns (5 points)
- Data is validated for user input when placing ships; see explanation in 2b. Row can be 0-3, column can be 0-4 (3 points)
- Ship cannot be placed where a ship already exists; see explanation in 2b. (3 points)
- Board is printed with 5 “S”s correctly placed based on user input after user has placed all 5 ships (3 points)
- Correct ship number (1-5) is printed after each ship is placed (3 points)
- Computers ships are randomly placed in computer board; see explanation in 2c (5 points)
- After ships are placed, user and computer take turns aiming at ships (3 points)
- Hit/miss are correctly printed after user/computer takes aim (3 points)
- If ship is hit, ‘X’ is added to board in correct place (3 points)
- If ship is missed ‘O’ is added to board in correct place (3 points)
- When computer board is printed, no ‘S’ are shown – only hit ships are displayed (‘X’) (2 points)
- When all 5 ships are hit – either user or computer – the game ends (3 points)
- Correct winner is printed (3 points)
- Correct rounds played is printed (3 points)